1. Manual Test Cases (Positive Test cases)

Test Case ID	Title	Preconditions	Steps	Test Data	Expected Result	Priority
TC_POS_01	Successful User Registration	User is on the registration page. Email not already registered.	1. Open registration page. 2. Enter valid Name. 3. Enter valid Email. 4. Enter valid Password. 5. Enter matching Confirm Password. 6. Click Register.	Name: ManasaUser Email: manasa@test.com Password: Password12345	User account created successfully. API returns **201 Created** and success message displayed.	High
TC_POS_02	Successful User Login	User already registered.	1. Open login page. 2. Enter registered email. 3. Enter correct password. 4. Click Login.	Email: manasa@test.com Password: Password12345	Login successful. JWT token generated and user redirected to dashboard.	High
TC_POS_03	Access Dashboard with Valid JWT	User logged in and JWT token generated.	1. Login successfully. 2. Navigate to `/dashboard`.	Valid JWT Token	Dashboard loads successfully and user data displayed.	High
TC_POS_04	Email Normalization (Uppercase Email)	Registration page opened.	1. Enter name. 2. Enter uppercase email. 3. Enter valid password and confirm password. 4. Click register.	Email: USER@MAIL.COM	Email converted to lowercase and registration successful.	Medium
TC_POS_05	Registration with Maximum Name Length	Registration page open.	1. Enter name with 24 characters. 2. Enter valid email. 3. Enter password and confirm password. 4. Click register.	Name: AAAAAAAAAAAAAAAAAAAAAAAA	Registration successful.	Medium
TC_POS_06	Login After Successful Registration	User newly registered.	1. Register user. 2. Navigate to login page. 3. Enter credentials. 4. Click login.	Email: newuser@test.com Password: Password12345	Login successful and dashboard displayed.	High

TC_POS_07	Access Protected API with JWT	User logged in.	1. Login to application. 2. Send request to protected endpoint `/dashboard` with JWT token.	Valid JWT	API returns **200 OK** and protected data returned.	High
TC_POS_08	Registration Requests within Rate Limit	Rate limit configured as 10 requests/hour.	1. Send registration requests up to 10 times within one hour.	Valid registration data	All requests processed successfully.	Medium
TC_POS_09	Login with Trimmed Email Spaces	User registered.	1. Enter email with leading/trailing spaces. 2. Enter correct password. 3. Click login.	Email: " user@test.com "	Spaces trimmed and login successful.	Low
TC_POS_10	Password Meeting Security Requirements	Registration page open.	1. Enter valid name. 2. Enter email. 3. Enter password containing letters and numbers. 4. Confirm password. 5. Click register.	Password: SecurePass12345	Registration successful.	High

Manual Test Cases (Negative Test cases)

Test Case ID	Title	Preconditions	Steps	Test Data	Expected Result	Priority
TC_NEG_01	Name Less Than Minimum Length	User is on the registration page. Email not already registered.	1. Open registration page. 2. Enter valid Name. 3. Enter valid Email. 4. Enter valid Password. 5. Enter matching Confirm Password. 6. Click Register.	Name: ManasaUser Email: manasa@test.com Password: Password12345	User account created successfully. API returns **201 Created** and success message displayed.	High
TC_NEG_02	Name Exceeds Maximum Length	User already registered.	1. Open login page. 2. Enter registered email. 3. Enter correct password. 4. Click Login.	Email: manasa@test.com Password: Password12345	Login successful. JWT token generated and user redirected to dashboard.	High

TC_NEG_03	Invalid Email Format	User logged in and JWT token generated.	1. Login successfully. 2. Navigate to `/dashboard`.	Valid JWT Token	Dashboard loads successfully and user data displayed.	High
TC_NEG_04	Password Less Than 12 Characters	Registration page opened.	1. Enter name. 2. Enter uppercase email. 3. Enter valid password and confirm password. 4. Click register.	Email: USER@MAIL.COM	Email converted to lowercase and registration successful.	Medium
TC_NEG_05	Password Without Numbers	Registration page open.	1. Enter name with 24 characters. 2. Enter valid email. 3. Enter password and confirm password. 4. Click register.	Name: AAAAAAAAAAAAAAAAAAAAAAAA	Registration successful.	Medium
TC_NEG_06	Confirm Password Mismatch	User newly registered.	1. Register user. 2. Navigate to login page. 3. Enter credentials. 4. Click login.	Email: newuser@test.com Password: Password12345	Login successful and dashboard displayed.	High
TC_NEG_07	Duplicate Email Registration	User logged in.	1. Login to application. 2. Send request to protected endpoint `/dashboard` with JWT token.	Valid JWT	API returns **200 OK** and protected data returned.	High
TC_NEG_08	Login with Incorrect Password	Rate limit configured as 10 requests/hour.	1. Send registration requests up to 10 times within one hour.	Valid registration data	All requests processed successfully.	Medium
TC_NEG_09	Login with Unregistered Email	User registered.	1. Enter email with leading/trailing spaces. 2. Enter correct password. 3. Click login.	Email: " user@test.com "	Spaces trimmed and login successful.	Low

TC_NEG_10	Access Dashboard Without Authentication	Registration page open.	1. Enter valid name. 2. Enter email. 3. Enter password containing letters and numbers. 4. Confirm password. 5. Click register.	Password: SecurePass12345	Registration successful.	High
TC_NEG_11	Concurrency Registration (Same Email)	System running normally.	1. Two users attempt registration simultaneously with same email.	Email: same@test.com	One request succeeds, second fails with **409 Conflict**.	High
TC_NEG_12	Invalid JWT Token	User logged in but token modified.	1. Send API request with tampered JWT token.	Invalid JWT	Request rejected with **401 Unauthorized**.	High
TC_NEG_13	Registration Rate Limit Exceeded	Rate limit configured.	1. Send more than 10 registration requests within one hour.	Valid registration data	11th request blocked with **429 Too Many Requests**.	High

2.Boundary Test Cases

Test Case ID	Title	Preconditions	Steps	Test Data	Expected Result
TC_BND_01	Name Length Below Minimum	User on registration page.	1. Open registration page 2. Enter name with 4 characters 3. Enter valid email and password 4. Click Register	Name: John	System shows validation error because minimum length is 5

TC_BND_02	Name Length Minimum Boundary	User on registration page	1. Open registration page 2. Enter name with exactly 5 characters 3. Fill other fields 4. Click Register	Name: James	Registration successful
TC_BND_03	Name Length Maximum Boundary	User on registration page	1. Enter name with exactly 24 characters 2. Fill other required fields 3. Click Register	Name: AAAAAAAAAAAAAAAAAAAAAAAA (24 chars)	Registration successful
TC_BND_04	Name Length Above Maximum	User on registration page	1. Enter name with 25 characters 2. Enter other valid fields 3. Click Register	Name: AAAAAAAAAAAAAAAAAAAAAAAAA (25 chars)	Validation error shown because max length is 24
TC_BND_05	Password Length Below Minimum	User on registration page	.1. Enter valid name and email 2. Enter password with 11 characters 3. Confirm password 4. Click Register	Password: Pass1234567 (11 chars)	Validation error because minimum password length is 12
TC_BND_06	Password Length Minimum Boundary	User on registration page	.1. Enter valid name and email 2. Enter password with exactly 12 characters 3. Confirm password 4. Click Register	Password: Pass12345678 (12 chars)	Registration successful
TC_BND_07	Email with Uppercase Letters	User on registration page	1. Enter valid name 2. Enter email in uppercase 3. Enter valid password 4. Click Register	Email: USER@MAIL.COM	System converts email to lowercase and registration succeeds
TC_BND_08	Email with Leading/Trailing Spaces	User on registration page	1. Enter valid name 2. Enter email with spaces before and after 3. Enter password 4. Click Register	Email: " user@mail.com "	Spaces trimmed and email processed correctly

TC_BND_09	Extremely Long Email	User on registration page	1. Enter valid name 2. Enter very long but valid email address 3. Enter password 4. Click Register	Email length close to 254 characters	System accepts email if within allowed limit
TC_BND_10	Registration Attempts at Rate Limit	User on registration page	.1. Send registration request 10 times within one hour	Valid registration data	All 10 requests processed successfully
TC_BND_11	Registration Attempts Above Rate Limit	User on registration page	.1. Send 11 registration requests within one hour	Valid registration data	11th request rejected with **429 Too Many Requests**

So we test:

- **Just below the boundary**
- **At the boundary**
- **Just above the boundary**

3. Test Data Sets

[Testcasesexcel1.csv]

4. Automation Testing (Using Selenium with Java)

Tools Used

- **Language:** Java
- **Automation Tool:** Selenium WebDriver
- **Browser:** Chrome

1. BaseTest.java

Purpose

This file contains the **common setup and teardown code** used by all test cases.

Why we use it

Instead of writing browser initialization in every test, we place it in **BaseTest.java** so all test files can reuse it.

What it does

- Starts the browser
- Maximizes the window
- Defines the base URL
- Closes the browser after tests

File: `BaseTest.java`

```
import org.openqa.selenium.WebDriver;

import org.openqa.selenium.chrome.ChromeDriver;

public class BaseTest {

    public static WebDriver driver;

    public static String baseUrl =
"https://the-internet.herokuapp.com";
```

```
    public static void setup() {

        driver = new ChromeDriver();

        driver.manage().window().maximize();

    }

    public static void tearDown() {

        driver.quit();

    }

}
```

---

2. SuccessfulRegistrationTest.java

Purpose

Automates **successful user registration scenario**.

Why it is in a separate file

Each test scenario should be independent so that tests are easier to maintain and debug.

File: `SuccessfulRegistrationTest.java`

```
import org.openqa.selenium.By;

import org.openqa.selenium.WebDriver;

import org.openqa.selenium.chrome.ChromeDriver;

public class SuccessfulRegistration {

public static void main(String[] args) throws InterruptedException {
```

```
        WebDriver driver = new ChromeDriver();

        driver.manage().window().maximize();

        driver.get("https://demowebshop.tricentis.com/register");

        // Select gender

        driver.findElement(By.id("gender-male")).click();

        // Enter first name

        driver.findElement(By.id("FirstName")).sendKeys("Manasa");

        // Enter last name

        driver.findElement(By.id("LastName")).sendKeys("Test");

        // Enter email

        driver.findElement(By.id("Email"))

                .sendKeys("manasa" + System.currentTimeMillis() + "@test.com");

        // Enter password

        driver.findElement(By.id("Password")).sendKeys("Password123");

        // Confirm password

        driver.findElement(By.id("ConfirmPassword")).sendKeys("Password123");

        // Click register button

        driver.findElement(By.id("register-button")).click();

        Thread.sleep(3000);

        System.out.println("Successful Registration Completed");

        driver.quit();

    }

}
```

---

3. DuplicateEmailRegistrationTest.java

Purpose

Tests system behavior when the user tries to **register with an email that already exists**.

Why this test is important

Duplicate email validation prevents **multiple accounts with the same email**.

File: `DuplicateEmailRegistrationTest.java`

```
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;

public class DuplicateEmailRegistrationTest {

    public static void main(String[] args) throws InterruptedException {

        WebDriver driver = new ChromeDriver();

        driver.manage().window().maximize();

        driver.get("https://demowebshop.tricentis.com/register");

        driver.findElement(By.id("gender-male")).click();

        driver.findElement(By.id("FirstName")).sendKeys("Manasa");

        driver.findElement(By.id("LastName")).sendKeys("User");

        // Email that was already registered earlier
        driver.findElement(By.id("Email")).sendKeys("manasa@test.com");

        driver.findElement(By.id("Password")).sendKeys("Password123");

        driver.findElement(By.id("ConfirmPassword")).sendKeys("Password123");

        driver.findElement(By.id("register-button")).click();

        Thread.sleep(3000);
```

```
        // Check error message
        String errorMessage =
            driver.findElement(By.className("message-error")).getText();

        System.out.println("Error Message: " + errorMessage);

        driver.quit();
    }
}
```

---

4. SuccessfulLoginTest.java

Purpose

Automates **valid login functionality**.

File: `SuccessfulLoginTest.java`

```
import org.openqa.selenium.By;

public class SuccessfulLoginTest extends BaseTest {

    public static void main(String[] args) throws InterruptedException
{

        setup();

        driver.get(baseUrl + "/login");

        driver.findElement(By.id("username")).sendKeys("tomsmith");

        driver.findElement(By.id("password"))

                .sendKeys("SuperSecretPassword!");

        driver.findElement(By.cssSelector("button[type='submit']"))

                .click();
```

```
        Thread.sleep(2000);

        System.out.println("Successful Login Test Executed");

        tearDown();

    }

}
```

---

5. InvalidLoginTest.java

Purpose

Checks system response when **wrong password is entered**.

File: `InvalidLoginTest.java`

```
import org.openqa.selenium.By;

public class Login extends BaseTest {

    public static void main(String[] args) throws InterruptedException
{

        setup();

        driver.get(baseUrl + "/login");

        driver.findElement(By.id("username")).sendKeys("tomsmith");

        driver.findElement(By.id("password"))

                .sendKeys("WrongPassword");

        driver.findElement(By.cssSelector("button[type='submit']"))

                .click();
```

```
        Thread.sleep(2000);

        System.out.println("Invalid Login Test Executed");
tearDown();

    }

}
```

Why this test is needed

To verify that the system **does not allow login with incorrect credentials**.

---

6. DashboardWithoutLoginTest.java

Purpose

Tests security behavior when accessing protected pages without authentication.

File: `DashboardWithoutLoginTest.java`

```
import org.openqa.selenium.By;

public class WithoutLogin extends BaseTest {

   public static void main(String[] args) throws InterruptedException {

      setup();

      driver.get(baseUrl + "/login");

      driver.findElement(By.cssSelector("button[type='submit']"))
            .click();

      Thread.sleep(2000);

      System.out.println("Successful WithoutLogin Test Executed");

      tearDown();
```

```
  }
}
```

5. Assumptions

The following assumptions were made while designing the test cases and automation scripts for the authentication system.

1. Environment Setup

- The application is assumed to run in a **local or staging environment** (e.g., `http://localhost:3000`) with the backend server and database properly configured.

- The testing environment includes a supported browser (e.g., **Google Chrome**) and required tools such as **Selenium WebDriver and Java** for automation.

2. JWT Authentication Behavior

- After a successful login, the system generates a **JSON Web Token (JWT)** that is used to authenticate subsequent requests to protected routes such as `/dashboard`.

- The JWT token is assumed to be stored securely in the **browser (e.g., cookies or local storage)** and included in API requests for authorization.

3. Token Expiration

- The JWT token is assumed to have a **limited validity period (for example, 30–60 minutes)** after which it expires.

- Once the token expires, the user must **log in again to obtain a new valid token** before accessing protected resources.

4. Validation Rules

- The application enforces the following input validation rules:

    - **Name length:** minimum 5 characters and maximum 24 characters

    - **Password length:** minimum 12 characters and must include both letters and numbers
    - **Email:** must follow a valid email format and must be unique in the system

5. Rate Limiting

- The system is assumed to enforce a **rate limit of 10 registration attempts per hour per user/IP**.
- If the limit is exceeded, the system returns an appropriate error such as **HTTP 429 – Too Many Requests**.